Bot de Video Vertical para TikTok

https://gemini.google.com/app/6a8d2837beb925c9

User prompt: Actúa como un Desarrollador Senior de Python experto en automatización, web scraping, procesamiento de video y APIs de redes sociales. Quiero que diseñes y crees un programa automatizado ('bot') para generar videos verticales (9:16) estilo 'Reddit Stories' para TikTok y YouTube Shorts. Tu tarea es crear un plan de desarrollo paso a paso y proporcionarme el código base inicial en Python para cada módulo. El flujo de trabajo que el programa debe seguir es: Módulo 1: Scraping de Reddit: Usar PRAW para extraer la historia "Top" del día de subreddits como r/AITA o r/HistoriasDeReddit. Necesito el título y el cuerpo del texto. Filtra por longitud (ej. entre 500 y 1000 caracteres). Módulo 2: Texto a Voz (TTS): Usar la librería edge-tts (gratuita) para convertir el título y el texto en archivos de audio MP3 con una voz española natural. Módulo 3: Procesamiento de Fondo: Elegir un clip de gameplay aleatorio de una carpeta local (ej. de 10 minutos de Minecraft o GTA V que yo ya tenga), y recortar un segmento aleatorio con la misma duración exacta que el audio generado. Módulo 4: Montaje de Video: Usar MoviePy para superponer el audio sobre el video de fondo, ajustar el video a formato vertical (1080x1920) y agregar subtítulos palabra por palabra (estilo karaoke) usando Whisper (de OpenAI, localmente si es posible) para generar los tiempos de las palabras. Módulo 5: Orquestador: Un script principal (main.py) que ejecute todo el proceso secuencialmente. Lo que necesito de ti: A. Una lista de las librerías de Python exactas que debo instalar con pip install. B. Una explicación estructurada de cómo debo organizar las carpetas del proyecto. C. El código Python funcional y comentado para los Módulos 1 y 2 (Scraping y TTS), ya que son los más sencillos para empezar, junto con el script orquestador básico. D. Consejos sobre cómo configurar las credenciales de la API de Reddit. Empecemos por la estructura y los primeros módulos para asegurarnos de que funciona antes de pasar al montaje de video, que es más complejo.

Response: Este es un proyecto excelente y una de las automatizaciones más rentables en cuanto a esfuerzo-resultado para creación de contenido. La arquitectura que propones es sólida. Al separar el scraping, la generación de voz, y el procesamiento de video en módulos distintos, te aseguras de que si una API falla (o si quieres cambiar el subreddit o la voz), no tengas que reescribir todo el bot.

Vamos a sentar las bases del proyecto, organizarlo como lo haría un equipo profesional, y dejar funcionando los dos primeros eslabones de la cadena.

A. Dependencias del Proyecto

Abre tu terminal y ejecuta el siguiente comando para instalar las librerías exactas que necesitaremos para todo el flujo:

```
pip install praw edge-tts moviepy openai-whisper python-dotenv
```

- **`praw`**: El wrapper oficial de la API de Reddit.
- **`edge-tts`**: Interfaz para el motor neuronal de texto a voz de Microsoft Edge (alta calidad y sin límites).
- **`moviepy`**: Para el recorte de clips y superposición de capas (audio, video, texto).
- **`openai-whisper`**: Para la transcripción local y obtención de marcas de tiempo exactas (timestamps) de cada palabra.
- **`python-dotenv`**: Para cargar variables de entorno y no exponer tus contraseñas en el código.

B. Estructura de Carpetas

Para mantener el código escalable, te recomiendo esta estructura exacta. Créala antes de añadir el código:

```
reddit_video_bot/
├── .env                # Aquí irán tus credenciales de Reddit (no lo subas a GitHub)
├── main.py             # Módulo 5: Orquestador del flujo
├── modules/
│   ├── __init__.py     # Archivo vacío para que Python reconozca la carpeta
│   ├── scraper.py      # Módulo 1: Conexión a Reddit
│   ├── tts.py          # Módulo 2: Generación de voz
│   └── video.py        # Módulos 3 y 4: Montaje (lo haremos en la siguiente fase)
├── assets/
│   ├── backgrounds/    # Guarda aquí tus gameplays de Minecraft/GTA (ej. gta_1.mp4)
│   └── fonts/          # Fuentes .ttf para los subtítulos (ej. Montserrat-Bold.ttf)
└── output/
    ├── audio/          # Audios temporales generados por edge-tts
    └── video/          # Los Shorts/TikToks finales
```

C. Código Base (Módulos 1, 2 y 5)

Aquí tienes el código inicial. He envuelto la librería asíncrona de `edge-tts` para que puedas usarla fácilmente de manera síncrona en tu orquestador.

1. `modules/scraper.py`

Este script se conecta a Reddit, busca en el top del día y filtra por longitud.

```
import praw
import os

def obtener_historia_reddit(subreddit_name="HistoriasDeReddit", limite_caracteres=(500, 1000)):
    # Inicializa el cliente usando las credenciales del entorno
    reddit = praw.Reddit(
        client_id=os.getenv("REDDIT_CLIENT_ID"),
        client_secret=os.getenv("REDDIT_CLIENT_SECRET"),
        user_agent=os.getenv("REDDIT_USER_AGENT")
```

```
    )

    subreddit = reddit.subreddit(subreddit_name)

    # Buscar en los 15 mejores posts del día
    for submission in subreddit.top(time_filter="day", limit=15):
        # Evitar posts fijados (anuncios de moderadores) o posts sin texto
        if submission.stickied or not submission.selftext:
            continue

        texto_completo = f"{submission.title}. {submission.selftext}"

        # Validar longitud
        if limite_caracteres[0] <= len(texto_completo) <= limite_caracteres[1]:
            return {
                "id": submission.id,
                "titulo": submission.title,
                "texto": submission.selftext,
                "texto_completo": texto_completo
            }

    raise ValueError(f"No se encontró ninguna historia entre {limite_caracteres[0]} y {limite_caracteres[1]} caracte
```

2. `modules/tts.py`

Convierte el texto extraído a un archivo MP3 usando una voz española de alta calidad.

```
import edge_tts
import asyncio

async def _generar_audio_async(texto, ruta_salida, voz):
    """Función asíncrona interna para interactuar con edge-tts."""
    communicate = edge_tts.Communicate(texto, voz)
    await communicate.save(ruta_salida)

def crear_audio_tts(texto, ruta_salida, voz="es-ES-AlvaroNeural"):
    """
    Wrapper síncrono.
    Voces recomendadas: 'es-ES-AlvaroNeural' (España Hombre), 'es-MX-JorgeNeural' (México Hombre).
    """
    # Ejecuta el loop de eventos de asyncio
    asyncio.run(_generar_audio_async(texto, ruta_salida, voz))
    return ruta_salida
```

3. `main.py`

El núcleo de tu bot que orquesta los primeros módulos.

```
import os
from dotenv import load_dotenv
from modules.scraper import obtener_historia_reddit
from modules.tts import crear_audio_tts

def main():
    # Cargar variables de entorno desde el archivo .env
    load_dotenv()

    # Asegurar que los directorios de salida existen
    os.makedirs("output/audio", exist_ok=True)
    os.makedirs("output/video", exist_ok=True)

    print("Iniciando bot de Reddit...")

    # 1. Scraping
    print("-> Buscando la mejor historia del día...")
    try:
        historia = obtener_historia_reddit("HistoriasDeReddit")
        print(f"   [OK] Historia encontrada: {historia['titulo']}")
    except Exception as e:
        print(f"   [ERROR] en scraping: {e}")
        return

    # 2. Text-to-Speech (TTS)
    ruta_audio = f"output/audio/{historia['id']}.mp3"
    print(f"-> Generando audio en: {ruta_audio}")
    try:
        crear_audio_tts(historia["texto_completo"], ruta_audio)
        print("   [OK] ¡Audio generado con éxito!")
    except Exception as e:
```

```
        print(f"   [ERROR] generando TTS: {e}")
        return

    print("\nFase 1 completada. Listo para procesar el video.")

if __name__ == "__main__":
    main()
```

D. Configuración de Credenciales (Reddit API)

Para que PRAW funcione, necesitas registrar tu bot en Reddit.

1. **Acceder al panel de desarrolladores**

   Inicia sesión en Reddit desde el navegador web y ve a reddit.com/prefs/apps.

2. **Crear nueva aplicación**

   Desplázate hasta abajo y haz clic en el botón **"Are you a developer? Create an app..."**.

3. **Configurar como Script**

   Ponle un nombre a tu bot, y asegúrate de seleccionar el radio button que dice **script** (esto es vital, de lo contrario la autenticación fallará). En `redirect uri` puedes poner `http://localhost:8080`.

4. **Extraer las credenciales**

   Una vez creada, verás un texto bajo el nombre de tu bot (ese es el `Client ID`) y un campo llamado `secret`.

En la raíz de tu proyecto (junto a `main.py`), crea un archivo llamado **exactamente** `.env` y añade estas líneas (reemplazando con tus datos):

```
REDDIT_CLIENT_ID=tu_client_id_aqui
REDDIT_CLIENT_SECRET=tu_secret_aqui
REDDIT_USER_AGENT=windows:reddit_video_bot:v1.0 (by /u/TuUsuarioDeReddit)
```

User prompt: mira esto

Response: ¡Excelente trabajo estructurando la base del proyecto! El código que has proporcionado es muy limpio y sigue las mejores prácticas (separación de responsabilidades, uso de variables de entorno y un orquestador central).

Veo que tu `config.py` ya define los directorios `GAMEPLAY_DIR` y `OUTPUT_DIR`, y tu `main.py` está perfectamente preparado para inyectar el audio en los módulos siguientes. También he notado que tus dependencias de la Fase 2 ya están listadas en el `requirements.txt`.

Para implementar la **Fase 2** (Módulos 3 y 4), vamos a tomar una decisión de diseño importante para **optimizar el rendimiento**: En lugar de que el Módulo 3 recorte el vídeo de fondo y lo guarde en el disco (lo cual consumiría mucho tiempo de renderizado I/O), el Módulo 3 se encargará exclusivamente de encontrar un clip válido, y el Módulo 4 hará el recorte dinámico en la memoria RAM antes de componerlo con el audio y los subtítulos.

A continuación, tienes el código para implementar estos dos módulos.

Módulo 3: Selección de Fondo (`modules/video_processor.py`)

Este módulo examina tu carpeta de `gameplay`, lee la duración del audio generado y busca aleatoriamente un vídeo que sea lo suficientemente largo para cubrir la historia.

```
import os
import random
from moviepy.editor import AudioFileClip, VideoFileClip

from config import GAMEPLAY_DIR

def get_random_background_clip(audio_path: str) -> str:
    """
    Selecciona un vídeo de fondo aleatorio que sea más largo que el audio.
    No recorta el vídeo aquí para ahorrar I/O; eso se hace en memoria en el Módulo 4.
    """
    # 1. Obtener la duración exacta del audio generado
    try:
        with AudioFileClip(audio_path) as audio:
            audio_duration = audio.duration
    except Exception as e:
        raise RuntimeError(f"Error al leer el audio {audio_path}: {e}")

    # 2. Listar vídeos disponibles en el directorio configurado
    valid_extensions = (".mp4", ".mov", ".avi")
    videos = [f for f in os.listdir(GAMEPLAY_DIR) if f.lower().endswith(valid_extensions)]

    if not videos:
```

```
            raise FileNotFoundError(
                f"No se encontraron vídeos de gameplay en {GAMEPLAY_DIR}. "
                "Asegúrate de copiar algunos clips allí."
            )

        # 3. Buscar aleatoriamente un vídeo que cumpla con la duración requerida
        random.shuffle(videos)
        for video_name in videos:
            video_path = os.path.join(GAMEPLAY_DIR, video_name)
            try:
                with VideoFileClip(video_path) as clip:
                    if clip.duration > audio_duration:
                        return video_path
            except Exception as e:
                print(f"⚠️ Error al leer el vídeo {video_name}: {e}")
                continue

        # Si terminamos el bucle sin retornar, ningún vídeo es válido
        raise ValueError(
            f"Ningún vídeo en {GAMEPLAY_DIR} es lo suficientemente largo "
            f"(se necesitan al menos {audio_duration:.2f} segundos)."
        )
```

Módulo 4: Montaje y Subtítulos (`modules/video_editor.py`)

Este es el cerebro de la Fase 2. Aquí usamos **Whisper** para generar los tiempos exactos de las palabras y **MoviePy** para reencuadrar el vídeo a vertical (9:16), superponer el audio e inyectar el texto estilo "karaoke".

```
import os
import random
import whisper
from moviepy.editor import VideoFileClip, AudioFileClip, TextClip, CompositeVideoClip
from moviepy.video.fx.all import crop, resize

from config import OUTPUT_DIR

def create_final_video(audio_path: str, background_clip_path: str, story: dict = None) -> str:
    """
    Monta el vídeo final: transcribe, recorta a 9:16, sincroniza audio/texto y renderiza.
    """
    # 1. Cargar el audio y el modelo de Whisper (el modelo "base" es rápido y decente)
    print("   [4.1] Cargando Whisper y transcribiendo audio...")
    audio = AudioFileClip(audio_path)
    model = whisper.load_model("base")

    # word_timestamps=True es vital para el estilo "karaoke"
    transcription = model.transcribe(audio_path, word_timestamps=True, language="es")

    # 2. Cargar y preparar el clip de vídeo de fondo en memoria
    print("   [4.2] Preparando y reencuadrando vídeo de fondo (9:16)...")
    bg_clip = VideoFileClip(background_clip_path)

    # Seleccionar un punto de inicio aleatorio
    max_start_time = bg_clip.duration - audio.duration
    start_time = random.uniform(0, max_start_time)
    bg_clip = bg_clip.subclip(start_time, start_time + audio.duration)

    # Recortar a relación de aspecto 9:16 (vertical) centrado
    w, h = bg_clip.size
    target_ratio = 9 / 16
    current_ratio = w / h

    if current_ratio > target_ratio:
        # Vídeo muy ancho (ej. 16:9), recortamos los lados
        new_w = int(h * target_ratio)
        bg_clip = bg_clip.fx(crop, x_center=w/2, y_center=h/2, width=new_w, height=h)
    else:
        # Vídeo muy alto, recortamos arriba y abajo
        new_h = int(w / target_ratio)
        bg_clip = bg_clip.fx(crop, x_center=w/2, y_center=h/2, width=w, height=new_h)

    # Opcional: Escalar a 1080x1920 exactos para estándar de TikTok/Shorts
    bg_clip = bg_clip.fx(resize, newsize=(1080, 1920))
    bg_clip = bg_clip.set_audio(audio)

    # 3. Generar los clips de texto (Subtítulos)
    print("   [4.3] Generando subtítulos sincronizados...")
    text_clips = []
```

```
    # Extraer segmentos y palabras de Whisper
    for segment in transcription.get("segments", []):
        for word_info in segment.get("words", []):
            word_text = word_info["word"].strip()
            start = word_info["start"]
            end = word_info["end"]

            if not word_text:
                continue

            # Crear el clip de texto para la palabra actual
            # NOTA: Requiere ImageMagick instalado en el sistema
            txt_clip = TextClip(
                word_text,
                fontsize=90,
                color='white',
                font='Arial-Bold', # Puedes cambiarlo por una fuente de assets/fonts/
                stroke_color='black',
                stroke_width=4,
                method='caption',
                align='center'
            )

            # Sincronizar y posicionar (centrado en la pantalla)
            txt_clip = (txt_clip
                        .set_start(start)
                        .set_end(end)
                        .set_position(('center', 'center')))

            text_clips.append(txt_clip)

    # 4. Componer y Renderizar
    print("   [4.4] Renderizando composición final (esto puede tardar)...")
    final_video = CompositeVideoClip([bg_clip] + text_clips)

    # Generar nombre de salida basado en el ID de la historia[cite: 9]
    story_id = story.get("id", "output") if story else "output"
    final_filename = f"{story_id}_final.mp4"
    final_path = str(OUTPUT_DIR / final_filename)

    # Escribir el archivo usando los codecs adecuados para RRSS
    final_video.write_videofile(
        final_path,
        fps=30,
        codec="libx264",
        audio_codec="aac",
        logger=None # Cambia a 'bar' si quieres ver la barra de progreso en consola
    )

    # 5. Limpieza de memoria
    audio.close()
    bg_clip.close()
    final_video.close()

    return final_path
```

⚠️ Dos notas críticas de infraestructura

Como mencionas en tu `README.md`, hay dos requisitos del sistema operativo que debes verificar para que la Fase 2 no lance errores crípticos:

1. **ImageMagick (Vital para MoviePy):** La clase `TextClip` lo requiere estrictamente. Tienes que descargarlo e instalarlo. Si estás en Windows, asegúrate de marcar la casilla *"Install legacy utilities (mogrify.exe, convert.exe)"* durante la instalación. Si MoviePy no lo detecta automáticamente, tendrás que apuntar la ruta en la configuración de la librería.
2. **PyTorch & Whisper:** Whisper usará la CPU por defecto, lo que puede hacer que la transcripción sea algo lenta. Si tienes una tarjeta gráfica dedicada y quieres acelerarlo, asegúrate de desinstalar el PyTorch estándar e instalar la versión específica con soporte para CUDA.

Has planteado los cimientos técnicos de forma impecable. Ahora que tenemos el pipeline completo capaz de emitir palabras sueltas, ¿te gustaría que mejoremos la estética de los subtítulos (por ejemplo, mostrando la oración completa pero resaltando la palabra actual en otro color) o prefieres probar primero que este ensamblaje renderiza sin problemas de entorno en tu máquina?

User prompt: Voy a montar el proyecto completo (estructura + Módulos 1, 2 y el orquestador) y te dejo la explicación de cada punto que pediste. Estructuró módulos faltantes para completar arquitectura del proyecto Estructuró módulos faltantes para completar arquitectura del proyecto Crear la estructura de carpetas del proyecto Crear config.py con la configuración central del proyecto Crear __init__.py del paquete modules Crear Módulo 1: scraper.py (extracción de historias de Reddit con PRAW) Crear Módulo 2: tts.py (texto a voz con edge-tts) Creating placeholder stubs for the remaining modules. Crear placeholder del Módulo 3 (fondo de vídeo) para que main.py no rompa al importarlo Crear placeholder del Módulo 4 (montaje de vídeo) para que main.py no rompa al importarlo Crear el orquestador principal main.py (Módulo 5) Listo

Ahora los archivos de configuración y documentación: Se crearon 4 archivos Se crearon 4 archivos Crear requirements.txt Crear plantilla .env.example para las credenciales Crear .gitignore Crear README con instrucciones de instalación y uso Listo Verifico que todo compila correctamente antes de entregarlo: Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración Aquí tienes el proyecto completo en reddit_video_bot.zip, con los Módulos 1, 2 y el orquestador ya funcionales, y los Módulos 3/4 como placeholders (lanzan NotImplementedError de forma controlada) para que main.py no rompa mientras seguimos desarrollando. A. Librerías a instalar bash pip install praw edge-tts python-dotenv# Para la Fase 2 (ya incluidas en requirements.txt, no hace falta instalarlas todavía):pip install moviepy openai-whisper torch ffmpeg-python También necesitarás ffmpeg instalado a nivel de sistema (no es paquete pip): sudo apt install ffmpeg en Linux, brew install ffmpeg en Mac, o el binario oficial en Windows. MoviePy y Whisper dependen de él. B. Estructura de carpetas reddit_video_bot/ ├── main.py # Orquestador (Módulo 5) ├── config.py # Credenciales, rutas y constantes centralizadas ├── requirements.txt ├── .env.example ├── .gitignore ├── modules/ │ ├── scraper.py # Módulo 1 ✅ │ ├── tts.py # Módulo 2 ✅ │ ├── video_processor.py # Módulo 3 ⏳ (placeholder) │ └── video_editor.py # Módulo 4 ⏳ (placeholder) ├── assets/ │ ├── audio/ # MP3 generados │ ├── gameplay/ # ⚠️ copia aquí tus clips de Minecraft/GTA │ └── output/ # Vídeos finales (Fase 2) └── data/ └── used_posts.json # Histórico para no repetir historias La idea de config.py es que ningún módulo tenga valores "hardcodeados" — todo (voz, rango de longitud, subreddits, rutas) se ajusta en un solo sitio. C. Qué hace el código incluido modules/scraper.py: recorre SUBREDDITS en orden aleatorio, mira el "top del día" de cada uno, descarta posts fijados o sin texto, filtra por longitud (500–1000 caracteres), y guarda un histórico en data/used_posts.json para no repetir historias entre ejecuciones. modules/tts.py: usa edge_tts.Communicate (asíncrono) para narrar "título + pausa + cuerpo" en un solo MP3. Incluye --list-voices para ver todas las voces en español disponibles y elegir la que más te guste. main.py: encadena 1 → 2, y ya deja preparadas las llamadas a los Módulos 3 y 4 (que ahora solo imprimen un aviso y se detienen, sin romper el flujo). Pruébalo así: bash python modules/scraper.py # solo scrapingpython modules/tts.py # solo TTSpython main.py # pipeline completo (Fase 1) D. Configurar las credenciales de Reddit Entra a https://www.reddit.com/prefs/apps con tu cuenta de Reddit. Clic en "create app" (o "create another app") al final de la página. Rellena:name: cualquier nombre, ej. RedditStoryBot Tipo: elige script (es el que necesitas para PRAW con usuario/contraseña de app, uso personal/no interactivo) redirect uri: pon algo como http://localhost:8080 (no se usará realmente, pero el campo es obligatorio) Al crear la app verás:client_id: la cadena corta justo debajo del nombre de la app client_secret: la cadena etiquetada como "secret" Copia .env.example a .env y rellena: REDDIT_CLIENT_ID=tu_client_id REDDIT_CLIENT_SECRET=tu_client_secret REDDIT_USER_AGENT=RedditStoryBot/1.0 by tu_usuario_de_reddit El user_agent debe ser único y descriptivo — Reddit puede limitarte si usas un user-agent genérico o el de otra app. 6. .env ya está en .gitignore, así que no se subirá por accidente si usas git. Dos avisos prácticos para cuando pases a producción: Reddit tiene términos de uso específicos para su API (incluyendo límites/tarifas para uso a gran volumen o comercial); si el bot empieza a monetizar en TikTok/YouTube conviene revisar la política vigente de la Reddit Data API. Considera añadir siempre una referencia al subreddit/post original en la descripción del vídeo — además de ser buena práctica, mucha audiencia de "Reddit stories" lo espera. Cuando confirmes que el scraping y el audio funcionan bien en tu entorno, seguimos con el Módulo 3 (recorte de gameplay) y el Módulo 4 (montaje + subtítulos karaoke con Whisper), que es donde entra la parte más pesada de MoviePy.Voy a montar el proyecto completo (estructura + Módulos 1, 2 y el orquestador) y te dejo la explicación de cada punto que pediste. Estructuró módulos faltantes para completar arquitectura del proyecto Estructuró módulos faltantes para completar arquitectura del proyecto Crear la estructura de carpetas del proyecto Crear config.py con la configuración central del proyecto Crear __init__.py del paquete modules Crear Módulo 1: scraper.py (extracción de historias de Reddit con PRAW) Crear Módulo 2: tts.py (texto a voz con edge-tts) Creating placeholder stubs for the remaining modules. Crear placeholder del Módulo 3 (fondo de vídeo) para que main.py no rompa al importarlo Crear placeholder del Módulo 4 (montaje de vídeo) para que main.py no rompa al importarlo Crear el orquestador principal main.py (Módulo 5) Listo

Ahora los archivos de configuración y documentación: Se crearon 4 archivos Se crearon 4 archivos Crear requirements.txt Crear plantilla .env.example para las credenciales Crear .gitignore Crear README con instrucciones de instalación y uso Listo Verifico que todo compila correctamente antes de entregarlo: Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración Aquí tienes el proyecto completo en reddit_video_bot.zip, con los Módulos 1, 2 y el orquestador ya funcionales, y los Módulos 3/4 como placeholders (lanzan NotImplementedError de forma controlada) para que main.py no rompa mientras seguimos desarrollando. A. Librerías a instalar bash pip install praw edge-tts python-dotenv# Para la Fase 2 (ya incluidas en requirements.txt, no hace falta instalarlas todavía):pip install moviepy openai-whisper torch ffmpeg-python También necesitarás ffmpeg instalado a nivel de sistema (no es paquete pip): sudo apt install ffmpeg en Linux, brew install ffmpeg en Mac, o el binario oficial en Windows. MoviePy y Whisper dependen de él. B. Estructura de carpetas reddit_video_bot/ ├── main.py # Orquestador (Módulo 5) ├── config.py # Credenciales, rutas y constantes centralizadas ├── requirements.txt ├── .env.example ├── .gitignore ├── modules/ │ ├── scraper.py # Módulo 1 ✅ │ ├── tts.py # Módulo 2 ✅ │ ├── video_processor.py # Módulo 3 ⏳ (placeholder) │ └── video_editor.py # Módulo 4 ⏳ (placeholder) ├── assets/ │ ├── audio/ # MP3 generados │ ├── gameplay/ # ⚠️ copia aquí tus clips de Minecraft/GTA │ └── output/ # Vídeos finales (Fase 2) └── data/ └── used_posts.json # Histórico para no repetir historias La idea de config.py es que ningún módulo tenga valores "hardcodeados" — todo (voz, rango de longitud, subreddits, rutas) se ajusta en un solo sitio. C. Qué hace el código incluido modules/scraper.py: recorre SUBREDDITS en orden aleatorio, mira el "top del día" de cada uno, descarta posts fijados o sin texto, filtra por longitud (500–1000 caracteres), y guarda un histórico en data/used_posts.json para no repetir historias entre ejecuciones. modules/tts.py: usa edge_tts.Communicate (asíncrono) para narrar "título + pausa + cuerpo" en un solo MP3. Incluye --list-voices para ver todas las voces en español disponibles y elegir la que más te guste. main.py: encadena 1 → 2, y ya deja preparadas las llamadas a los Módulos 3 y 4 (que ahora solo imprimen un aviso y se detienen, sin romper el flujo). Pruébalo así: bash python modules/scraper.py # solo scrapingpython modules/tts.py # solo TTSpython main.py # pipeline completo (Fase 1) D. Configurar las credenciales de Reddit Entra a https://www.reddit.com/prefs/apps con tu cuenta de Reddit. Clic en "create app" (o "create another app") al final de la página. Rellena:name: cualquier nombre, ej. RedditStoryBot Tipo: elige script (es el que necesitas para PRAW con usuario/contraseña de app, uso personal/no interactivo) redirect uri: pon algo como http://localhost:8080 (no se usará realmente, pero el campo es obligatorio) Al crear la app verás:client_id: la cadena corta justo debajo del nombre de la app client_secret: la cadena etiquetada como "secret" Copia .env.example a .env y rellena: REDDIT_CLIENT_ID=tu_client_id REDDIT_CLIENT_SECRET=tu_client_secret REDDIT_USER_AGENT=RedditStoryBot/1.0 by tu_usuario_de_reddit El user_agent debe ser único y descriptivo — Reddit puede limitarte si usas un user-agent genérico o el de otra app. 6. .env ya está en .gitignore, así que no se subirá por accidente si usas git. Dos avisos prácticos para cuando pases a producción: Reddit tiene términos de uso específicos para su API (incluyendo límites/tarifas para uso a gran volumen o comercial); si el bot empieza a monetizar en TikTok/YouTube conviene revisar la política vigente de la Reddit Data API. Considera añadir siempre una referencia al subreddit/post original en la descripción del vídeo — además de ser buena práctica, mucha audiencia de "Reddit stories" lo espera. Cuando confirmes que el scraping y el audio funcionan bien en tu entorno, seguimos con el Módulo 3 (recorte de gameplay) y el Módulo 4 (montaje + subtítulos karaoke con Whisper), que es donde entra la parte más pesada de MoviePy. *esto es lo que llevo hecho quiero que me digas que me falta y porque*

Response: Tienes una base excelente. La estructura de carpetas, la separación de responsabilidades y la forma en que has aislado la configuración en `config.py` es exactamente como se debe plantear un proyecto profesional para que sea escalable.

Sin embargo, basándome en el resumen que acabas de compartir, tu bot está funcionalmente estancado en la **Fase 1** (Scraping + Audio). El pipeline se corta justo antes de procesar el vídeo porque los módulos finales están vacíos.

Aquí tienes exactamente lo que te falta para tener el bot 100% operativo y por qué necesitas cada elemento:

1. Reemplazar los Placeholders de los Módulos 3 y 4

- **Lo que falta:** En el texto que me muestras, confirmas que `modules/video_processor.py` y `modules/video_editor.py` son solo *stubs* que lanzan `NotImplementedError`.

- **Por qué falta:** Sin la implementación real de estos módulos, el bot descarga la historia y genera el audio, pero se detiene ahí. Debes sustituir el código de esos archivos por los scripts con **MoviePy** y **Whisper** que te proporcioné en mi respuesta anterior. Esos scripts contienen la lógica real para recortar el vídeo en formato 9:16 e incrustar los subtítulos dinámicos.

2. Instalar ImageMagick en el Sistema Operativo

- **Lo que falta:** Aunque mencionas que necesitas FFmpeg (lo cual es correcto para procesar vídeo y audio), te falta instalar **ImageMagick**.
- **Por qué falta:** MoviePy utiliza la clase `TextClip` para generar los subtítulos de Whisper palabra por palabra. MoviePy no renderiza texto de forma nativa; utiliza ImageMagick por debajo para convertir cada palabra en una imagen con fondo transparente antes de superponerla al vídeo. Si omites este paso, el Módulo 4 crasheará inmediatamente con un error de binario no encontrado.

3. Poblar la carpeta `assets/gameplay/`

- **Lo que falta:** Añadir archivos `.mp4` (o `.mov`) reales a tu directorio de assets.
- **Por qué falta:** El orquestador y el Módulo 3 están diseñados para escanear esa carpeta y elegir un vídeo al azar que sea más largo que el audio generado. Si ejecutas la Fase 2 con esa carpeta vacía, el código lanzará un `FileNotFoundError` porque no tendrá lienzo sobre el que trabajar.

4. Instalar PyTorch (Preferiblemente con soporte CUDA)

- **Lo que falta:** En tus librerías mencionas `torch`, pero para usar Whisper de forma eficiente necesitas la versión correcta de PyTorch según tu hardware.
- **Por qué falta:** El Módulo 4 usa Whisper para escuchar el audio que generó Edge-TTS y mapear los milisegundos exactos de cada palabra. Si instalas PyTorch por defecto (solo CPU), la transcripción funcionará, pero el renderizado del vídeo tardará muchísimo más. Si tu equipo tiene una tarjeta gráfica dedicada (NVIDIA), instalar la versión de PyTorch con soporte CUDA reducirá el tiempo de procesamiento drásticamente.

¿Qué sistema operativo estás utilizando para desarrollar y ejecutar este bot, de modo que pueda darte las instrucciones exactas para configurar ImageMagick sin que MoviePy te dé problemas de rutas?

User prompt: esto?

Response: ¡Exacto! Has dado en el clavo.

Si te fijas en el código que me acabas de enviar, tanto en tu archivo `modules/video_editor.py` como en `modules/video_processor.py`, la única acción que realizan las funciones internamente es ejecutar un `raise NotImplementedError`.

Esos archivos son solo "carcasas" vacías que dejaste preparadas en la Fase 1. Cuando tu orquestador llega al paso 3, llama a esos módulos, choca contra ese error programado, imprime el aviso de que la Fase 2 no está implementada y detiene el proceso.

La solución es directa: Tienes que borrar el contenido actual de esos dos archivos y reemplazarlo con los bloques de código completos que te proporcioné en mi respuesta anterior.

- Tu nuevo `video_processor.py` se encargará de importar `moviepy` para leer la duración del audio y elegir el clip de fondo adecuado.
- Tu nuevo `video_editor.py` se encargará de cargar `whisper` para transcribir los tiempos y montar los subtítulos dinámicos.

Una vez que guardes esos cambios, la lógica de tu bot en Python estará 100% terminada. Sin embargo, para que esos dos módulos se ejecuten sin que el entorno tire errores de dependencias externas, necesitamos preparar tu sistema.

Para darte las instrucciones exactas de cómo instalar ImageMagick y configurar PyTorch (evitando problemas de rutas o renderizados extremadamente lentos), ¿qué sistema operativo utilizas en el equipo donde estás desarrollando el bot ahora mismo?

User prompt: haber ponte por partes quiero que me digas que llevo hecho que esta bien que esta mal y que me falta por hacer

Response: ¡Claro que sí! Vamos a hacer una radiografía exacta de tu proyecto para que sepas exactamente dónde estás parado.

Lo que llevas hecho (y está muy bien)

Has construido unos cimientos de nivel profesional. Esta estructura inicial te ahorrará muchos dolores de cabeza en el futuro:

- **Arquitectura modular:** Tienes el proyecto dividido lógicamente en carpetas (`modules/`, `assets/`, `data/`).
- **Configuración centralizada:** Tu archivo `config.py` es excelente porque concentra todas las variables, rutas y credenciales en un solo lugar.
- **Seguridad:** Estás utilizando `.env` para tus claves de la API de Reddit, asegurando que no se expongan públicamente.
- **Módulo 1 (Scraper):** Tu script `scraper_2.py` está perfecto. Se conecta a Reddit, filtra por longitud y guarda un registro en `used_posts.json` para no repetir contenido.
- **Módulo 2 (TTS):** Tu script `tts_2.py` funciona correctamente para conectarse a Edge-TTS y generar el audio de forma asíncrona.
- **Orquestador:** Tu `main.py` está bien planteado para encadenar la Fase 1 de forma fluida.

Lo que está "mal" (el bloqueo actual)

No es que haya código erróneo, sino que el bot está programado para detenerse a propósito en la Fase 2.

- **Los archivos de vídeo están vacíos:** Tanto `video_editor_2.py` como `video_processor.py` son solo "carcasas". Actualmente, su única instrucción es lanzar un `NotImplementedError`.
- **Freno en el orquestador:** Cuando `main.py` intenta ejecutar el paso 3, choca contra ese error y aborta el pipeline.

Lo que te falta por hacer (Pasos exactos)

Para que el bot procese vídeos verticales con subtítulos, debes ejecutar estos pasos en orden:

1. **Actualizar el código:** Borra el contenido de `video_editor_2.py` y `video_processor.py`. Pega en ellos los bloques de código con la lógica de MoviePy y Whisper que te proporcioné anteriormente.
2. **Añadir material rodante:** Descarga algunos clips largos de Minecraft, GTA V o Parkour y guárdalos dentro de tu carpeta `assets/gameplay/`. Sin ellos, el bot no tendrá vídeos para recortar.
3. **Instalar librerías de Python (Fase 2):** Abre tu terminal y ejecuta la instalación de las herramientas de vídeo listadas en tu `requirements.txt`: `pip install moviepy openai-whisper torch ffmpeg-python`.
4. **Instalar programas del sistema:** Whisper y MoviePy necesitan dos programas instalados a nivel de sistema operativo para funcionar: **FFmpeg** (para procesar los archivos multimedia) e **ImageMagick** (para renderizar los subtítulos de texto).

Para darte los comandos exactos de instalación de FFmpeg e ImageMagick sin causar conflictos de rutas, ¿qué sistema operativo tiene el ordenador donde estás montando el bot?

User prompt: es un windows 11

Response: Perfecto, configurar este entorno en Windows 11 tiene un par de trucos específicos, especialmente con ImageMagick y MoviePy.

Aquí tienes la guía paso a paso, diseñada específicamente para que Windows 11 no te dé dolores de cabeza con las rutas o los archivos binarios al procesar los subtítulos.

1. Instalar FFmpeg (Procesamiento de vídeo)

En Windows 11, la forma más rápida y limpia de instalar FFmpeg sin tener que configurar las variables de entorno a mano es usar el gestor de paquetes oficial de Windows (`winget`).

1. Abre tu terminal (Símbolo del sistema o PowerShell) como Administrador.
2. Ejecuta este comando:

   ```
   winget install "FFmpeg (Essentials Build)"
   ```

3. Una vez que termine, **cierra esa ventana de la terminal y abre una nueva**. Puedes comprobar que se instaló bien escribiendo `ffmpeg -version`.

2. Instalar ImageMagick (Crucial para los subtítulos)

MoviePy necesita este programa para crear el texto "karaoke", pero en Windows hay que marcar una casilla específica durante la instalación.

1. Ve a la página oficial de descargas de ImageMagick (imagemagick.org).
2. En la sección de Windows, descarga el primer archivo ejecutable (suele llamarse algo como `ImageMagick-7.X.X-Q16-HDRI-x64-dll.exe`).
3. Ejecuta el instalador. **ATENCIÓN AQUÍ:** Cuando llegues a la pantalla de "Select Additional Tasks" (Seleccionar tareas adicionales), debes marcar obligatoriamente la casilla que dice **"Install legacy utilities (e.g. convert)"**. Si no marcas esto, MoviePy no lo reconocerá.
4. Finaliza la instalación.

El truco de Windows para MoviePy: A veces, incluso instalándolo bien, MoviePy no encuentra ImageMagick en Windows. Para evitar que el bot se estrelle en el Módulo 4, añade esta línea al principio de tu archivo `main.py` o de `video_editor.py` (ajustando la ruta a la versión que se haya instalado en tu carpeta de Archivos de Programa):

```
import os
# Fuerzas a MoviePy a encontrar el programa
os.environ["IMAGEMAGICK_BINARY"] = r"C:\Program Files\ImageMagick-7.1.1-Q16-HDRI\magick.exe"
```

3. Acelerar Whisper con PyTorch (Opcional pero muy recomendado)

Por defecto, al hacer `pip install torch`, se instala la versión de procesador (CPU). Whisper funcionará, pero tardará bastante en sacar los subtítulos de un audio de un minuto.

Si tu ordenador de escritorio tiene una tarjeta gráfica **NVIDIA**, puedes hacer que vuele instalando la versión CUDA:

1. En tu terminal, desinstala el PyTorch normal: `pip uninstall torch`

2. Instala la versión acelerada por GPU ejecutando:

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

(Si usas gráfica AMD o procesador gráfico integrado, sáltate este paso y mantén el `torch` normal).

4. Último paso de código

Ahora sí, una vez instalados los programas de sistema operativo:

- Mete un par de vídeos largos de Minecraft o GTA V (en `.mp4`) dentro de la carpeta `assets/gameplay/`.
- Reemplaza el código vacío de `video_processor.py` y `video_editor.py` por los bloques de código funcionales que vimos antes.

¿Tienes vídeos preparados para la carpeta `gameplay` o quieres que te pase algún recurso de donde puedas descargar clips libres de derechos para empezar a probar?
